

INTERNSHIP PROJECT REPORT

Diya

An unattended meditation kiosk

[Your Name] · [Institution]

[Month Year – Month Year] · Guide: [Mentor Name]

THE BRIEF

A visitor walks up. Nobody is there to help them.

A museum kiosk has to identify a stranger, run a camera-guided meditation session, hand them a personal report, and reset itself — unattended, all day.

No staff present

No keyboard

No scanner hardware

Must never stay closed

SCOPE

Two teams, one thin seam

HARDWARE TEAM (IITH)

Cameras, servos, CV

RealSense and thermal capture, posture and gaze analysis, and the meditation-app that produces the report PDF.

MINE

Everything around it

The kiosk application, the identification system, the backend and web pages, packaging, deployment — and the contract between the two sides.

The seam is two primitives, not a protocol. I launch one shell script and read one PDF off disk. They rewrote their internals all summer without ever breaking my app.

THE HARD PART

How do you identify someone with no scanner?

Every obvious answer costs hardware: a QR reader, a card reader, a touchscreen keyboard that needs cleaning between strangers.

DESIGN EVOLUTION

Three attempts

01

Data in the QR

The printed code carried the visitor's details. Fully offline — but still needs a reader at the kiosk, and puts personal data on paper.

02

QR holds an id

The code became a lookup key and the kiosk fetched details over the API. Better privacy — the scanner problem remained.

03 · SHIPPED

Invert it

The **kiosk shows the QR** and the **visitor's phone scans it**. The phone is the scanner, camera and keyboard. Zero hardware.

THE DECISION THAT MATTERED

The phone is the scanner

The kiosk asks the backend for a session, renders the token as a QR, and polls. The visitor's own phone does the reading.

POST /api/sessions

GET /api/sessions/:token

A constraint produced a simpler design than a budget would have.

Welcome

Start

Register with your phone



Scan this code with your phone camera to register.

Waiting for you to register...

New code

or just enter your details

Your name

Email (Gmail)

Age

END TO END

Four stages, one API

STAGE 01

Identify

An admin roster gives each person a private link. They open it on their phone and scan the kiosk.

```
GET /api/people/:token
POST /api/claim
```

STAGE 02

Claim

The backend links that identity to the kiosk's open session. The polling kiosk advances by itself.

```
GET /api/sessions/:token
→ status claimed
```

STAGE 03

Session

The kiosk launches the CV pipeline and blocks. This boundary is deliberately **not** an API.

```
bash scripts/run1.sh
writes report.pdf
```

STAGE 04

Report

The newest PDF written after the session began is rendered in-app, then the kiosk resets.

```
PDFium → page images
```

Kiosk to Backend is the only network boundary — plain REST over HTTPS.

Kiosk to meditation-app is a local process and a file on disk. No protocol to break.

RUNNING SYSTEM

The whole flow, on screen

Welcome



Ananya Sharma
Email: ananya.sharma@example.com
Age: 29

Start

Please wait — running your session...

Register with your phone

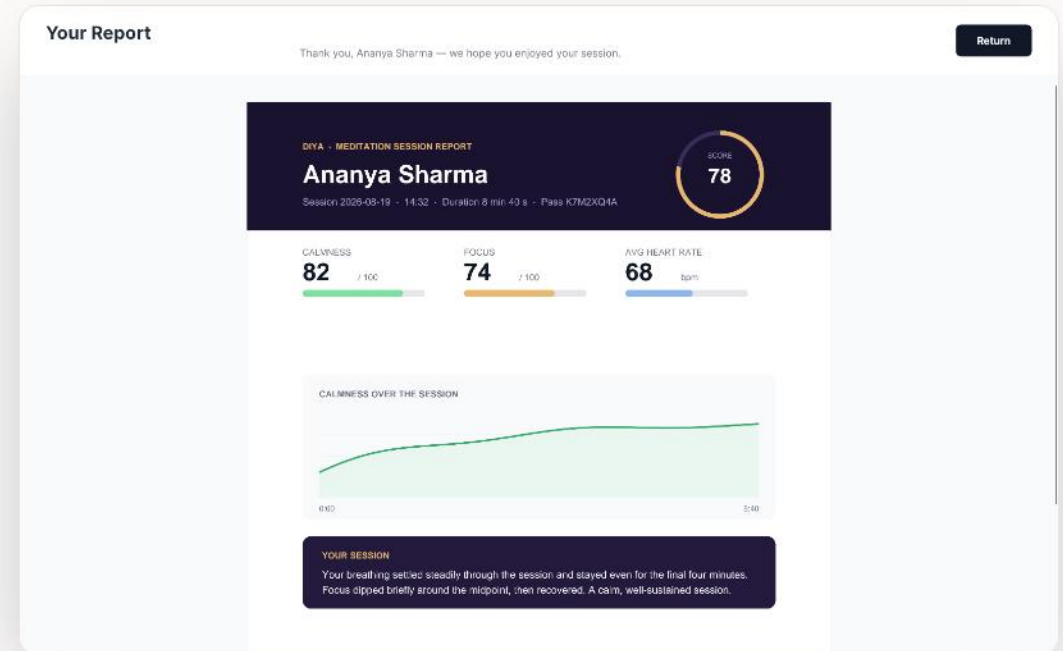


Calibrating cameras...

[New code](#)

or just enter your details

Authenticated. Name, email and roster photo arrive from the claim; the session starts with no button press.



Report. The PDF is decoded and rendered inside the app — no external viewer, no desktop visible.

UNDER THE HOOD

Three parts I built

KIOSK APP

C# · .NET 8 · Avalonia

The only .NET UI framework that officially supports Linux. Ships as a self-contained single-file .deb.

BACKEND

Node · Express · Postgres

Sessions, visitors and the admin roster. Serves the web pages from the same origin.

WEB

Three static pages

Phone registration, admin roster upload with in-browser XLSX parsing, and the per-person login scanner.

QRCoder

PDFtoImage / PDFium

SheetJS

html5-qrcode

systemd

Render

DEBUGGING

Three that cost me days

WAYLAND

Fullscreen refused

`CanResize="False"` sets fixed size hints, and the compositor then **refuses** to fullscreen the window. The intuition is backwards.

DEPENDENCIES

A clash that hides

Avalonia and the PDF renderer both use SkiaSharp. I verified they resolved to one version **before** committing to in-app rendering.

THE BEST ONE

"Can't reach the server"

The VM clock was in the past, so valid HTTPS certificates looked **not yet valid**. Nothing to do with the server at all.

TESTING

Building it without the hardware

For most of the internship there were no cameras and no meditation-app. The flow still had to be testable.

Two environment variables swap the pipeline for a mock. **No test code inside the app**, nothing extra in the shipped package — and those same variables are the real production override.

```
$ export DIYA_PIPELINE_SCRIPT=.../run1.mock.sh  
$ export DIYA_REPORT_DIR=/tmp/diya-reports  
$ diya-meditation
```

Calibrating cameras...

Running t3 (PDF report)...

All tasks completed successfully.

Login, session, report, thank-you, on any Linux desktop.

SHIPPING

One command each

BUILD

A single package

Self-contained .deb for amd64 and arm64.
The museum machine needs nothing pre-installed.

LOCK DOWN

A kiosk, not an app

systemd user service with Restart=always,
autologin, and GNOME escape shortcuts
disabled.

DEPLOY

Backend in one file

A blueprint provisions the web service and
Postgres together, then tracks main.

Plus documentation written for whoever comes next: an overview, a full command reference, an FAQ, and a handoff note recording the dead ends so nobody repeats them.

WHAT COMES NEXT

Where I handed it over

The full path works today: roster upload, phone login, automatic session, report on screen, reset. Four things are queued for whoever continues.

01

Personalise the report

Pass the visitor into the pipeline so the PDF is theirs, not generic.

02

Bound the pipeline

A timeout and a failure screen, so a dead camera cannot strand the kiosk.

03

Consent screen

Cameras observe visitors and identity data is stored. It needs asking first.

04

Real CV hardware

The interfaces are in place; the devices arrive after me.

Someone can walk up, log in with their phone, and leave with their report.

The vision hardware is the remaining milestone. Everything I could not finish is written down.

Thank you — questions?